

Reducing the problem

Let's look at this in terms of graph theory. We have $n \cdot m$ vertices corresponding to the squares of the grid and 1 vertex for the outside. We draw an edge from vertex u to v if it is possible to orient the arrow in u so that it points directly at v . (If a sign in a square is fixed, then there is exactly one outedge. If a direction of an arrow can be changed, then there are four outedges.)

Let's formulate the property of valid orientations of arrows (i.e. a valid evacuation plan). We want to select exactly one outedge for each vertex from the grid, in a manner that the resulting graph is acyclic. In particular, if the resulting graph is acyclic, following the selected outedges must eventually lead to a vertex with no outedge — the outside. We call this a *1-absorbance*.

The original problem asks us to find a min-weight 1-absorbance, for a suitable weighting of edges: the outedges corresponding to the original orientation of the arrow have weight 0 (costs us no reorientation), and all other edges have weight 1 (cost us 1 reorientation).

Important observation: finding a min-weight 1-absorbance is equivalent to finding a min-weight acyclic subgraph where all maximal paths lead outside (this we call a min-weight *absorbance*). Why is it so? Consider any absorbance that is not a 1-absorbance. Then it contains a vertex with multiple outedges. Then removing one of them does not change the acyclic property, nor the property that all maximal paths lead outside (as there is another way from that vertex), and can only reduce the total weight of edges, i.e. is never a worse solution.

Thus it is enough to find a min-weight absorbance. This can be solved using a standard algorithm — the Chu-Liu/Edmonds algorithm runs in $O(|E| \cdot |V|)$, and this can be speeded up to $O(|E| + |V| \log |V|)$ (Tarjan). We briefly describe how to achieve some speedup sufficient for solving all subtasks.

Note: this is the equivalent of min-weight spanning tree problem for directed graphs. One might be tempted to use the same algorithm. However, a simple greedy algorithm fails, since breaking the ties differently can lead to different-weight solutions (consider simply the case of $\text{``}>><\text{'}$

Chu-Liu/Edmonds refresher

Instead of solving the particular case where the weights are only 0 and 1, we can solve the general problem of finding a min-weight absorbance. Forget arrows, and consider an oriented weighted (with non-negative weights) graph $G = (V, E)$ with a designated sink vertex s (representing the outside vertex in our problem).

The first observation is, that without loss of generality, each non-sink vertex has an outedge with weight 0. Why is it so? Consider a graph G' on the same vertex and edge set, where for every vertex, the weight of its outedges were reduced by the minimal weight of its outedge in G . Let w be the sum of minimal outedge weights in G . Now if T is an absorbance in G with weight w_T , is it an absorbance in G' with weight $w_T - w$, as in a min-weight absorbance, every vertex will have exactly one outedge. Similarly, if T is an absorbance in G' with weight w_T , it is also an absorbance in G with weight $w_T + w$. Thus minimal absorbances are identical in both graphs.

Thus, assume that each non-sink vertex in G' has an outedge with weight 0. Now consider a following naive solution: pick a min-weight outedge per each non-sink vertex (breaking ties arbitrarily). If this is an absorbance, it has weight 0, thus must be a min-weight absorbance as all edges have non-negative weights.

If it is not, then it contains a cycle. Let C be a cycle in this solution. Now consider contracting C into a single vertex v_C , creating a new graph G'' with vertices $(V \setminus C) \cup \{v_C\}$, removing edges within the cycle C and replacing the edges with one endpoint in C , replace that endpoint with v_C .

Why is this new graph useful? We claim that a) the min-weight absorbance has the same weight in both G' and G'' and b) from a min-weight absorbance of G'' one can reconstruct a min-weight absorbance of G' .

To prove the first claim, we consider both sides of the inequality. Consider first a min-weight absorbance of G'' , T'' . One can create an absorbance T' of G' with the same weight as follows: expand v_C in T'' into the cycle C . This leads to a same-weight subgraph where all but one non-sink vertex have exactly one outedge. The vertex with two outedges is a cycle vertex, and one can easily verify that removing the cycle edge creates an absorbance.

Now consider a min-weight absorbance of G' , T' . Compress the nodes of C into a single vertex v_C . The resulting graph must be connected, with every vertex having a path to the sink. Hence we can remove some edges to obtain an absorbance of T'' , clearly with at most the weight of T' .

Combining the two inequalities concludes the proof. Thus we obtain a polynomial-time algorithm: create G' out of G by reweighting the edges, then select a min-weight outedge for each vertex. If this results in an absorbance, return it. Otherwise, find a cycle, compress it and recurse.

In each recursive call, we reduce the size of the graph by at least 1, thus we recurse at most $|V|$ times. Each recursive step can be implemented in linear time, thus giving us $O(|V| \cdot (|V| + |E|))$ time complexity, which translated to $O((nm)^2)$ time complexity in our original problem. This solution solves the small problem set.

Further details of the proof can be seen [here](https://www.cs.cmu.edu/~15850/notes/lec2.pdf) (<https://www.cs.cmu.edu/~15850/notes/lec2.pdf>).

Speeding up

The key observation on the way to the faster solution is, that the quadratic solution computes the same thing too many times. Namely, note that when compressing a cycle, we do not have to recalculate the selected outedge for vertices outside the cycle as the weights of their outedges do not change (thus neither does the set of min-weight outedges).

Thus instead of recursively contracting the cycles, the idea is to contract them iteratively: whenever a cycle is detected, compress it, processing the outedges going from it. If the min-weight outedge does not have a weight 0, reweight them. Then select a min-weight outedge from the cycle-vertex and proceed, detecting the cycles until we are left with an absorbance.

Doing this in naive way still leaves us with quadratic solution, we can utilize data structures to simulate these efficiently.

Detecting the cycles

To detect a cycle efficiently, we use a union-find-like structure. For each vertex, we remember not only the edge selected, but also where the path leads to. We update this information lazily: whenever we ask for an endpoint of the path we update this information on all vertices in the path, just like in union-find.

Suppose we are adding an edge $v \rightarrow w$. Then compare where the path from v and w leads to. If they have the same path-endpoints, then adding this edge creates a cycle.

Once we add a cycle, we want to avoid having to update the information for each vertex that might have a path of selected outedges leading to this cycle (as there might be linearly many). To avoid having to touch them, simply add a new endpoint for each vertex in the cycle: to the cycle-vertex itself. Note that since every vertex is compressed into a cycle at most once (thus leading to at most $2|V|$ vertices), doing $O(\log |G|)$ operations per vertex in detected cycle is still sufficiently fast.

And how do we go over all vertices in a cycle? We need a second union-find-like structure to see what cycle a particular vertex ended up (because by the time we process an edge, a vertex in question might have ended up compressed). Hence whenever we create a new cycle, all vertices of the cycle end up being connected (since we are using a union-find like structure, this operation can be done in $O(\log |G|)$ amortized time).

Then, when finding a cycle, we follow the edges, finding which cycle the vertex in question ended up in (or whether it is not in any yet).

Compressing the cycles

Once we find a cycle, we need to find a way to efficiently process its edges. We already shown how to deal with the edges going to the cycle (using the union-find-like structure). Now consider the edges going from the cycle. First of all, processing all is too much, as a vertex can end up being in (nested) cycles linearly many times. Hence we need something faster.

The idea is to use priority queues. For each vertex, keep its edges in a priority queue. For each priority queue, we can also remember a “lazy” flag — how much the edges’ cost should be decreased (the cost of the min-weight outedge).

Now, when compressing a cycle, we *merge* the prospective priority queues. Doing this naively is still slow, however one might use small-to-large, taking care of the “lazy” flags when merging. This leads to $O(\log^2 n)$ amortized time per merge.

Another solution, is to have a custom-made heap that supports merge in $O(\log n)$ time (such as binomial, leftist or another ad-hoc heap). This solution is a bit faster and if implementing it as lazy flags (such as in a segment tree) can save some implementation details when handling decrementing the weight of the outedges.

One then has to take care of a few implementation details, such as removing the edges that go between the vertices of the cycle. This can be done on-the-fly when selecting the min-weight out-edge from a (cycle)vertex: simply use the union-find-like structure for detecting cycles, and if an edge is between the vertices of the same cycle erase it, possibly decreasing the weights of the remaining edges in the priority queue.

Calculating the cost

With these two details in mind, we have an algorithm: go over all vertices in some order (such as adding them to a queue or a stack). For each vertex pick a min-weight outgoing edge. If that edge does not create a cycle, continue on a next vertex. If that edge creates a cycle, detect and compress it as described above, add it to the data structure of our choice to be processed, and continue on the next vertex.

Once we processed all vertices, and no cycle is outstanding, we get a weight of the min-weight absorbance: following the proof outline above, one can see that the weight of the min-weight absorbance is simply the sum by which the outedges’ costs were decreased (in total, over all, including the cycle vertices). This can be kept track of as we proceed with the outlined algorithm in $O(nm \cdot \log^c(nm))$ (where c is 1 or 2, depending on the priority queues used). This solves the second test case.

Reconstructing the solution

In order to deconstruct the valid absorbance for the original graph (and thus reorientation of the arrows), we need to do additional work. In particular once we find a min-weight absorbance for the resulting graph (with cycles contracted), we need to expand the cycles efficiently to get an absorbance of the same weight (thus a min-absorbance) for the original graph, by following the proof/construction outline above.

The idea is to process the cycles in reverse order in which they were compressed. Start with an initial absorbance, consisting of outedges of vertices that did not ended up in (further) cycles — note this may include some of the cycle vertices. To expand the cycle, first take an outedge from this cycle in the current absorbance. We have to keep information of which vertex this edge originally belonged to, and set this edge to be its outedge. Next add all the edges of the cycle that are not from this vertex to the new (modified) absorbance.

The trickiest part is to maintain the edge’s ancestry. One way is to go from the bottom: for each edge remember what was its original edge in the uncompressed (full) graph, and then go over all cycles where the edge’s origin belonged to, and set that once it is uncompressed, that vertex in question should not have a cycle-edge included. With a careful analysis, once can see that this only adds linear overhead, thus resulting in the same complexity as the algorithm calculating only the minimal cost.